

Partitionnement, shuffles et collecte

Module 6 — Partitions, skew, cache, broadcast, accumulateurs et formats

Objectifs pédagogiques

- * Comprendre le rôle et l'impact du partitionnement sur les performances Spark
- * Utiliser `repartition()` et `coalesce()` à bon escient
- * Identifier et corriger un problème de déséquilibre de données (data skew)
- * Maîtriser les niveaux de persistance : `cache()` et `persist()`
- * Choisir le bon format de fichier pour l'écriture distribuée
- * Collecter les données vers le Driver de manière sûre et efficace

Le partitionnement

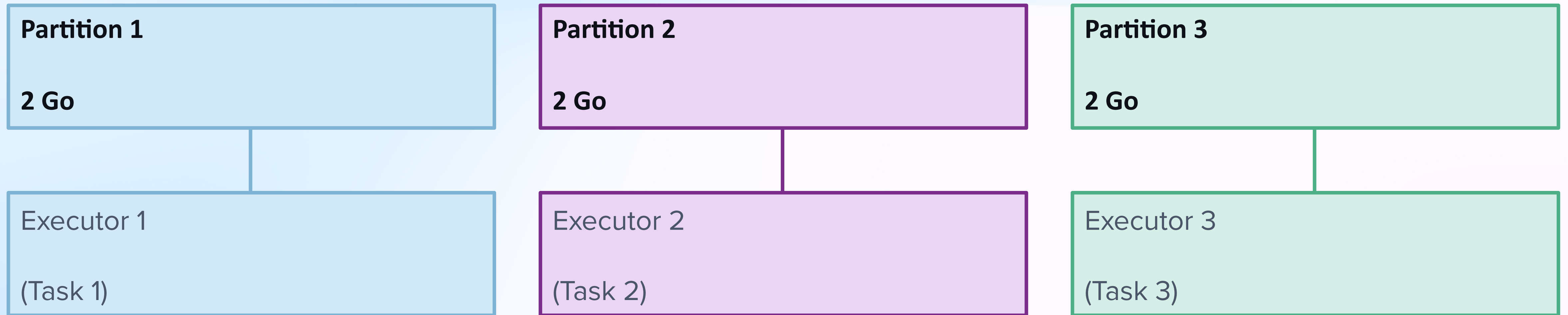
Fondement du calcul distribué

La partition — unité de parallélisme

1 partition = 1 Task = 1 cœur CPU

Le nombre de partitions détermine le degré de parallélisme :

DataFrame 12 Go — 6 partitions (2 vagues de 3)



La partition — unité de parallélisme

Règles pratiques

```
df.rdd.getNumPartitions()      # nombre de partitions courant
```

```
# Règle : 2 à 4 partitions par cœur → 10 Executors × 4 cœurs = 40 cœurs → 80 à 160 partitions
```

```
spark.conf.get("spark.sql.shuffle.partitions") # 200 par défaut ⚠ souvent trop élevé !
```

```
spark.conf.set("spark.sql.shuffle.partitions", "16") # préférer 2-4× les cœurs disponibles
```

Taille cible par partition : 128 Mo à 256 Mo. Pour 10 Go : $10\,000/128 \approx 78$ partitions. Trop peu = sous-utilisation ; trop = overhead de scheduling.

Sources et création des partitions

— Lecture de fichiers (1 partition $\approx$ 1 bloc HDFS = 128 Mo) —————

```
df = spark.read.parquet("data/") # nb partitions = nb fichiers × taille / 128 Mo
```

— Parallélisation explicite —————

```
rdd = sc.parallelize(range(1000), numSlices=8) # exactement 8 partitions
```

— Après un shuffle (groupBy, join, repartition...) —————

```
# Contrôlé par spark.sql.shuffle.partitions (200 par défaut)
```

— Lecture JDBC avec partitionnement explicite —————

```
df = spark.read.jdbc(url, table,  
    numPartitions=10, partitionColumn="id",  
    lowerBound=0, upperBound=1_000_000)
```

— Inspecter la distribution des partitions —————

```
df.rdd.mapPartitionsWithIndex(  
    lambda i, it: [(i, sum(1 for _ in it))]  
).toDF(["partition", "nb_lignes"]).orderBy("partition").show()
```


Sources et création des partitions

- * Un fichier unique = une seule partition : un CSV de 10 Go en un seul fichier crée 1 partition → 1 seule Task → pas de parallélisme !
- * Toujours appliquer `repartition(N)` après la lecture pour distribuer le travail sur le cluster.

Les fonctions de repartition et de fusion

repartition() et coalesce()

repartition() vs coalesce() — Le tableau de décision

repartition(N)

- Shuffle complet réseau
- Données équilibrées (même taille)
- Peut augmenter ou réduire
- Coûteux — transfert réseau
- Hash partitioning par colonne possible

coalesce(N)

- Fusion locale (pas de shuffle)
- Peut être déséquilibré
- Ne peut que réduire
- Peu coûteux
- Idéal avant une écriture

Besoin	Solution
Augmenter le nombre de partitions	repartition() — obligatoire
Diminuer, données équilibrées	coalesce() — moins coûteux
Diminuer, données déséquilibrées	repartition() — pour ré-équilibrer
Avant une écriture (moins de fichiers)	coalesce() — évite le shuffle

⚠ **coalesce(1)** : tout converge vers 1 Executor → goulot d'étranglement ! Préférer coalesce(4-10) ou maxRecordsPerFile.

repartition() en pratique

repartition() — cas d'usage

Fichier unique → paralléliser

```
df = spark.read.csv("gros_fichier.csv") # 1 partition
```

```
df = df.repartition(32)
```

Hash partitioning par colonne

```
df.repartition(8, F.col("ville"))
```

Avant un join intensif

```
df.repartition(100, "client_id")\  
  .join(df2, "client_id")
```

partitionBy sur RDD (HashPartitioner)

```
rdd_pair.partitionBy(4).cache()
```

○ Divise le fichier en 32 partitions

○ Divise le fichier en 8 partitions en
préservant la cohérence des villes

○ Divise le fichier, puis opère la
jointure

○ Partition d'un RDD

coalesce() en pratique

1 partition = 1 fichier

```
# coalesce() — cas d'usage
```

```
# Avant écriture → moins de fichiers
```

```
df.coalesce(10).write.parquet("output/")
```

```
# coalesce(1) → goulot !
```

```
df.coalesce(1).write.csv("output.csv")
```

```
# Contrôle fin du nb de fichiers
```

```
df.write\  
  .option("maxRecordsPerFile", 1_000_000)\  
  .parquet("output/")
```

```
# Repartition + partitionBy
```

```
df.repartition(F.col("pays"))\  
  .write.partitionBy("pays")\  
  .parquet("output/")
```

Réduit le nombre de fichier à créer avant l'écriture sur disque

Chaque partition est écrite dans un fichier séparé

Contrôle du nombre de fichiers, ici en fonction du nombre de lignes

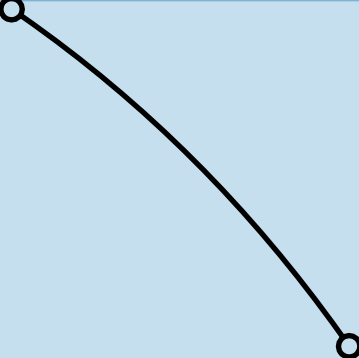
Un fichier par pays, dans des dossiers séparés

Pour tester les partitions

glom()

<<

glom() — outil de diagnostic : **rdd.glom().collect()** regroupe les éléments de chaque partition en une liste — permet de visualiser exactement ce que contient chaque partition et détecter un déséquilibre.



Ne fonctionne pas avec les DataFrame, qui doivent être convertis en RDD

Le déséquilibre des partitions

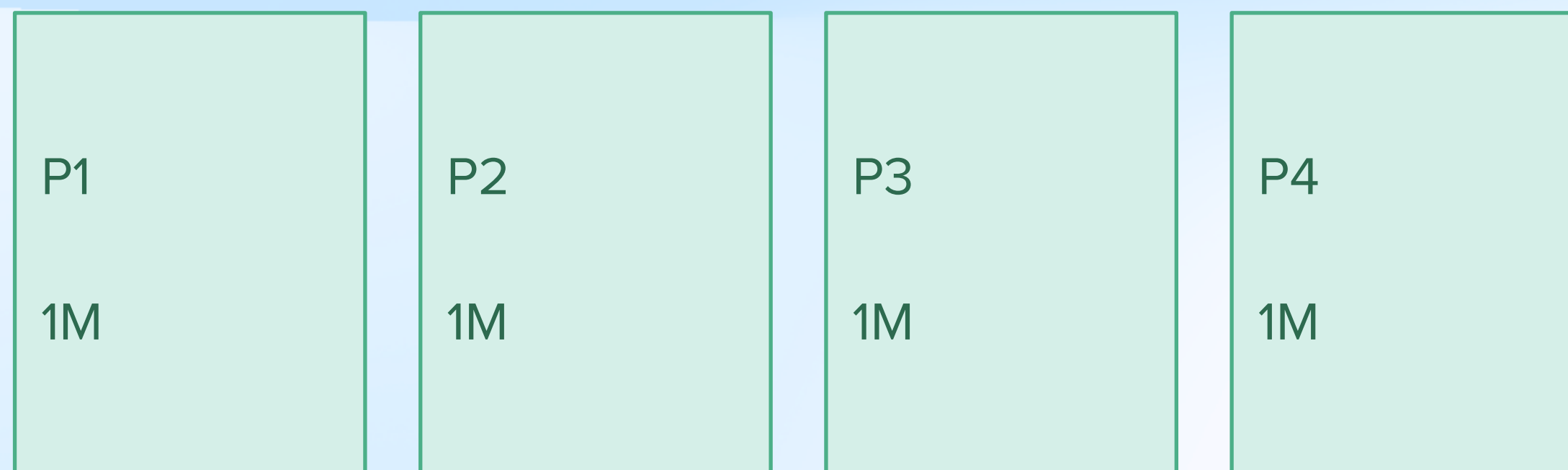
Ou « Data skew »

Data Skew

Définition

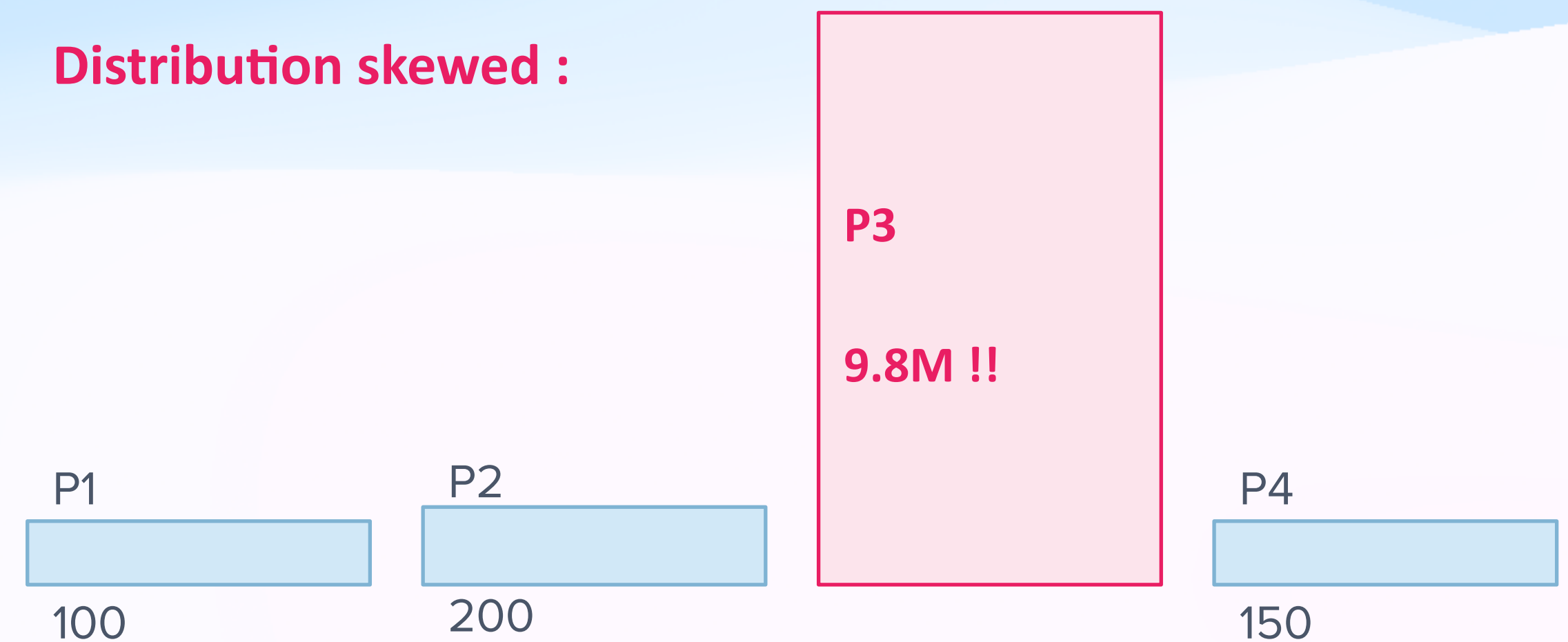
Certaines partitions beaucoup plus grandes → stragglers qui bloquent tout le stage

Distribution équilibrée :



~10 min ✓

Distribution skewed :



~50 min ✗ (bloquée par P3)

Data Skew

Détection

```
# Distribution des tailles de partitions
```

```
df.rdd\  
  .mapPartitionsWithIndex(lambda i, it: [(i, sum(1 for _ in it))])\  
  .toDF([« partition », "count"])\  
  .orderBy(« count »)\  
  .show()  # Vérifier les valeurs dominantes d'une clé de jointure
```

```
df.groupBy("client_id").count().orderBy(F.col("count").desc()).show(10)
```

Corriger le Data Skew

Technique 1 — Salting (ajout d'un sel aléatoire)

```
N_SALT = 10
```

```
# Grande table : sel aléatoire
```

```
df_large = df_large\  
    .withColumn(  
        « ville_salt»,  
        F.concat(F.col("ville"), F.lit("_"), (F.rand()*N_SALT).cast(« int").cast("string")  
    ))
```

```
# Petite table : répliquer chaque ligne N_SALT fois
```

```
df_ref_rep = df_ref\  
    .withColumn(« sel», F.explode(F.array([F.lit(i) for i in range(N_SALT)])))\  
    .withColumn("ville_salt", F.concat(F.col("ville"), F.lit("_"), F.col(« sel »).cast(« string »)))\  
    .drop("sel")
```

```
df_joint = df_large.join(df_ref_rep, "ville_salt").drop("ville_salt")
```


Corriger le Data Skew

Technique 2 — AQE (Spark 3.x)

```
spark.conf.set("spark.sql.adaptive.enabled","true")  
spark.conf.set("spark.sql.adaptive.skewJoin.enabled","true")  
# Partitions > 5× la médiane → découpées automatiquement
```

Corriger le Data Skew

Technique 3 — Broadcast join

```
df_large.join(broadcast(df_ref_skewee), "ville")  
# Si la table skewed est assez petite (< 10 Mo)
```

Corriger le Data Skew

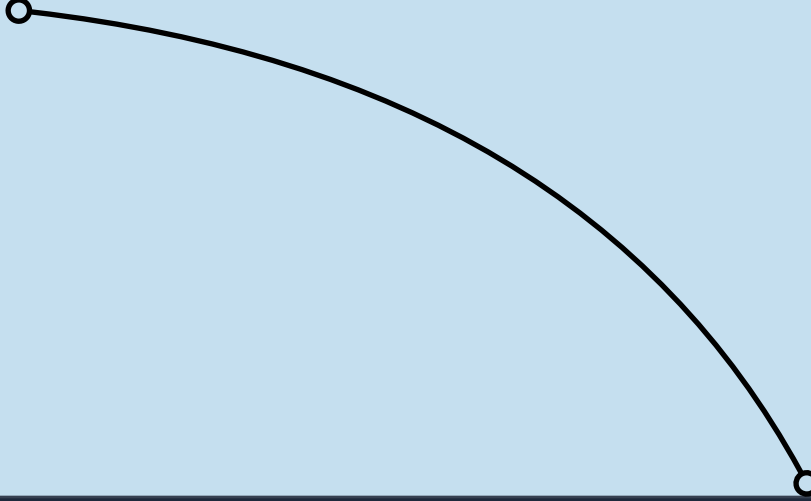
Technique 4 — Clés chaudes séparées

```
df_chaud = df.filter(F.col("ville").isin(["Paris","Lyon"]))  
resultat_chaud = df_chaud.join(broadcast(df_ref), "ville")  
resultat_final = resultat_chaud.union(df_froid.join(df_ref,"ville"))
```


Persistance et cache

cache() et persist() — Pourquoi et comment

Sans persistance, chaque action recalcule tout le DAG depuis le début



```
# Sans cache : df_prepare recalculé 3 fois (lecture + filtre + groupBy × 3)
```

```
df_prepare.show()          # Recalcul complet
```

```
df_prepare.count()         # Recalcul complet
```

```
df_prepare.write.parquet(...) # Recalcul complet
```

```
# Avec cache : calculé 1 fois, stocké, puis réutilisé
```

```
df_prepare.cache()
```

```
df_prepare.count()
```

```
# Déclenche le calcul ET le stockage
```

```
df_prepare.show()          # Depuis le cache → très rapide
```

Niveaux de persistance

Niveau	RAM	Disque	Sérialisé	Usage
MEMORY_ONLY	✓	✗	✗	RDD petits, RAM abondante
MEMORY_AND_DISK	✓	✓	✗	Défaut cache() — bonne tolérance
MEMORY_ONLY_SER	✓	✗	✓	Économise la RAM
MEMORY_AND_DISK_SER	✓	✓	✓	Équilibre RAM/disque
DISK_ONLY	✗	✓	✓	Grands datasets, RAM limitée
MEMORY_ONLY_2	✓	✗	✗	Haute disponibilité (répliqué ×2)
OFF_HEAP	✓	✗	✓	Évite le GC Java (Tungsten)


```
from pyspark import StorageLevel
```

Bonnes pratiques de cache et libération

```
#  Toujours déclencher une action après cache() pour matérialiser
```

```
df.cache()
```

```
df.count()
```

```
# Force le calcul ET le stockage en mémoire
```

```
#  persist() pour choisir le niveau explicitement
```

```
df.persist(StorageLevel.MEMORY_ONLY)      # RAM abondante
```

```
df.persist(StorageLevel.MEMORY_AND_DISK)   # ← cache() = ce niveau
```

```
df.persist(StorageLevel.DISK_ONLY)        # RAM très limitée
```

```
#  Libérer dès que le cache n'est plus utile
```

```
df_temp.cache()
```

```
# ... utilisation ...
```

```
df_temp.unpersist()  # Libération explicite
```

```
spark.catalog.clearCache()  # Vider tout le cache de la session
```

```
#  Checkpoint : casse un long lignage et matérialise
```

```
sc.setCheckpointDir("/tmp/spark-checkpoint")
```

```
rdd_long.checkpoint()      # Matérialise ET coupe le lignage
```


Bonnes pratiques de cache et libération

Quand cacher ? Uniquement les DataFrames (1) calculés par un DAG long/coûteux ET (2) utilisés plusieurs fois. Ne pas cacher systématiquement — le cache consomme de la RAM et peut évincer d'autres données.

Spark UI

→ onglet Storage : visualiser tous les DataFrames/RDD en cache, leur niveau, la fraction de RAM utilisée, et les partitions stockées.

Collecte des données vers le Driver

Collecte vers le Driver

Actions

```
# — Actions de collecte —  
df.collect()    # ⚠ TOUTES les lignes → OOM  
df.take(n)      # N premières lignes  
df.first()      # 1ère ligne seulement  
df.show(n)      # affiche, ne retourne rien  
df.toPandas()   # ⚠ tout en Pandas → OOM  
  
# Collecte scalaire  
total = df.count()  
somme = df.agg(F.sum("montant")).collect()[0][0]  
maxi = df.agg(F.max("montant")).first()[0]
```

Les opérations de collecte rapatrient des données du cluster vers le Driver — à utiliser avec précaution

Collecte vers le Driver

Sécurité

```
# — Collecte sécurisée —————
```

```
# Toujours limiter
```

```
df.filter(...).limit(10_000).collect()
```

```
df.sample(fraction=0.01).collect() # 1%
```

```
# Vérifier avant de collecter
```

```
n = df.count()
```

```
if n < 100_000:
```

```
    rows = df.collect()
```

```
else:
```

```
    print(f"Trop grand : {n} lignes")
```

```
# toPandas() → toujours limiter
```

```
df.filter(...).limit(5_000).toPandas()
```

Collecte vers le Driver

Collecte d'une seule colonne

```
valeurs = [row["ville"] for row in df.select("ville").collect()]  
valeurs = df.select("ville").rdd.flatMap(lambda x: x).collect()
```

Règle d'or : `collect()` et `toPandas()` sans `limit()` sur un grand DataFrame :
OutOfMemoryError garanti sur le Driver.

Préférer `write()` pour sauvegarder de gros volumes.

```
code_postal = {"Paris": "75", ...}
```

```
acc_err = sc.accumulator(0)
```

```
acc_nulls = sc.accumulator(0)
```

Variables broadcast et accumulateurs

 Du Driver vers le cluster (broadcast) et du cluster vers le Driver (accumulateurs)

```
cp_bc = sc.broadcast(code_postal)
```

```
@F.udf(StringType())
```

```
def get_code(ville):
```

```
    return cp_bc.value.get(ville)
```

```
df.withColumn("code", get_code(F.col("ville")))
```

```
def verifier(row):
```

```
    if row["montant"] is None:
```

```
        acc_nulls.add(1)
```

```
    elif row["
```

```
montant"] < 0:
```

```
        acc_err.add(
```

```
1)
```

 **Accumulateurs** : mis à jour uniquement lors des actions. En cas de re-exécution d'une tâche (tolérance aux pannes), l'accumulateur peut être incrémenté plusieurs fois → valeurs approchées dans les transformations. Utiliser uniquement dans des actions pour des valeurs exactes.

```
cp_bc.unpersist()
```

```
cp_bc.destroy()
```

```
df.rdd.filter(verifier).count() # action!
```


Écriture distribuée et formats de fichiers

API d'écriture et modes

df.write — syntaxe fluide commune à tous les formats

```
df.write\  
  .format("parquet")\  
  .mode("overwrite")\  
  .option("compression", "snappy")\  
  .partitionBy("pays", "annee")\  
  .save("chemin/vers/output/")  
  
# Raccourcis  
df.write.parquet("output/")  
df.write.csv("output/", header=True, sep=";")  
df.write.json("output/")  
df.write.orc("output/")
```

API d'écriture et modes

Les 4 modes d'écriture

Mode	Comportement si la destination existe
"error" (défaut)	Lance une exception
"overwrite"	Écrase les données existantes
"append"	Ajoute aux données existantes
"ignore"	Ne fait rien (silencieux)

Formats de fichiers — comparatif

CSV

Simple mais inefficace

- ✓ Universel, lisible par tout outil
- ✓ Compatible Excel, pandas, SQL
- ✗ Pas de schéma embarqué
- ✗ Pas de compression native
- ✗ Pas de partition pruning
- ✗ 5-10× plus lourd que Parquet

JSON

Flexible mais verbeux

- ✓ Hiérarchique, semi-structuré
- ✓ Facile à lire/déboguer
- ✗ Très verbeux (répétition des clés)
- ✗ Plus lent que Parquet
- ✗ JSON Lines : 1 objet par ligne

Parquet

★ *Format de référence production*

- ✓ Colonnaire (lit seules les colonnes nécessaires)
- ✓ Compression 5-10× vs CSV
- ✓ Schéma embarqué + predicate pushdown
- ✓ Partitionnement natif
- ✗ Moins lisible hors Spark

ORC

Optimisé pour Hive

- ✓ Comparable à Parquet
- ✓ Meilleur pour les requêtes Hive
- ✗ Moins répandu en dehors de l'écosystème Hive

Delta Lake

ACID sur Parquet

- ✓ Transactions ACID
- ✓ Time travel (versions)
- ✓ MERGE, UPDATE, DELETE
- ✗ Nécessite delta-spark installé
- ✗ Overhead de métadonnées

Structure générée :

output/ventes/pays=France/annee=2023/part-00000-xxx.parquet

output/ventes/pays=France/annee=2024/part-00000-xxx.parquet

output/ventes/pays=Allemagne/annee=2023/part-00000-xxx.parquet

Partitionnement physique — partitionBy() à l'écriture

Partition pruning : Spark lit UNIQUEMENT les sous-dossiers pertinents

```
df_lu.filter((F.col("pays")=="France")&(F.col("annee")==2024))
```

→ lit uniquement output/ventes/pays=France/annee=2024/

Choisir les colonnes de partition avec soin : trop de valeurs distinctes (ex: user_id) → des milliers de sous-dossiers → problème des petits fichiers. Préférer des colonnes à faible cardinalité : pays, année, mois, catégorie.

Contrôler le nombre de fichiers de sortie

```
df.coalesce(10).write.parquet("output/") # 10 fichiers, sans shuffle
```

```
df.repartition(10).write.parquet("output/") # 10 fichiers équilibrés, avec shuffle
```

```
df.repartition(F.col("pays")).write.partitionBy("pays").parquet("output/") # 1 fichier / pays
```

```
df.write.option("maxRecordsPerFile", 1_000_000).parquet("output/") # max 1M lignes / fichier
```


Structure générée :

output/ventes/pays=France/annee=2023/part-00000-xxx.parquet

output/ventes/pays=France/annee=2024/part-00000-xxx.parquet

output/ventes/pays=Allemagne/annee=2023/part-00000-xxx.parquet

Partitionnement physique — partitionBy() à l'écriture

Partition pruning : Spark lit UNIQUEMENT les sous-dossiers pertinents

```
df_lu.filter((F.col("pays")==France)&(F.col("annee")==2024))
```

→ lit uniquement output/ventes/pays=France/annee=2024/

Choisir les colonnes de partition avec soin : trop de valeurs distinctes (ex: user_id) → des milliers de sous-dossiers → problème des petits fichiers. Préférer des colonnes à faible cardinalité : pays, année, mois, catégorie.

Contrôler le nombre de fichiers de sortie

```
df.coalesce(10).write.parquet("output/") # 10 fichiers, sans shuffle
```

```
df.repartition(10).write.parquet("output/") # 10 fichiers équilibrés, avec shuffle
```

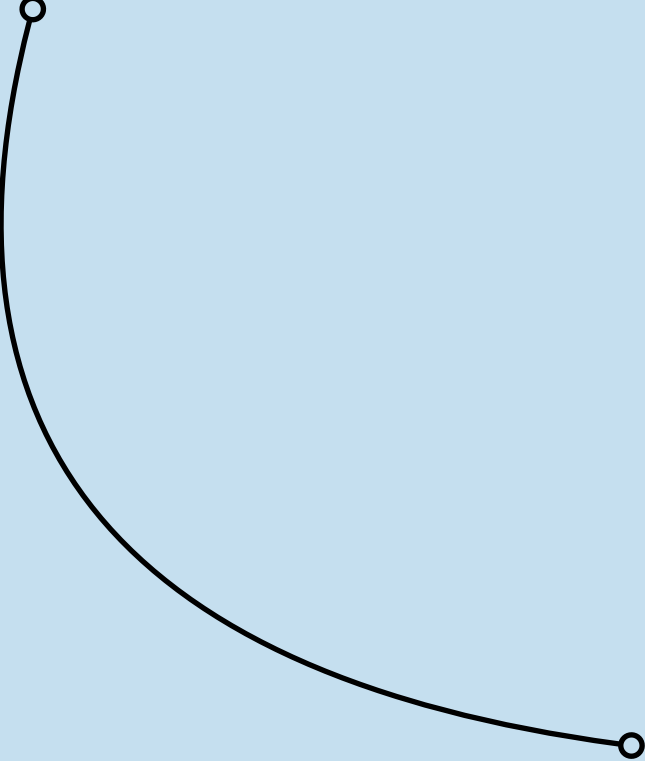
```
df.repartition(F.col("pays")).write.partitionBy("pays").parquet("output/") # 1 fichier / pays
```

```
df.write.option("maxRecordsPerFile", 1_000_000).parquet("output/") # max 1M lignes / fichier
```

Partitionnement physique

partitionBy() à l'écriture

*Organise les fichiers en sous-répertoires
par valeur de colonne
→ partition pruning à la lecture*



```
# Écriture partitionnée
df.write.partitionBy("pays", "annee").parquet("output/ventes/")

# Structure générée :
# output/ventes/pays=France/annee=2023/part-00000-xxx.parquet
# output/ventes/pays=France/annee=2024/part-00000-xxx.parquet
# output/ventes/pays=Allemagne/annee=2023/part-00000-xxx.parquet

# Partition pruning : Spark lit UNIQUEMENT les sous-dossiers pertinents
df_lu.filter((F.col("pays")== "France") & (F.col("annee")== 2024))
# → lit uniquement output/ventes/pays=France/annee=2024/
```

Résumé

Concept	Points clés à retenir
Partition	Unité de parallélisme — 1 partition = 1 Task = 1 cœur CPU
Nombre de partitions	2-4× le nombre de cœurs — shuffle.partitions = 200 par défaut (souvent trop élevé)
repartition()	Shuffle complet — peut augmenter ou réduire, équilibré, coûteux
coalesce()	Fusion locale — ne peut que réduire, sans shuffle, idéal avant écriture
Data Skew	Partitions déséquilibrées → stragglers — corriger avec salting, AQE, broadcast ou split
cache()	Raccourci MEMORY_AND_DISK — toujours suivre d'une action pour matérialiser
persist()	Contrôle fin du niveau : MEMORY_ONLY, DISK_ONLY, OFF_HEAP, répliqué...
Broadcast variable	Partage une petite structure Python vers tous les Executors (1 copie / Executor)
Accumulateur	Collecte des métriques des Executors vers le Driver — valeurs approchées en transformation
Parquet	Format de référence en production — colonnaire, compressé, predicate pushdown
collect() / toPandas()	⚠ Rapatrie tout au Driver — toujours limiter avec limit() ou sample()

Récapitulatif des outils de diagnostic

Spark UI → onglet Storage — Visualiser tous les DataFrames/RDD en cache, leur niveau et la fraction de RAM

Spark UI → onglet Stages — Shuffle Write/Read, Spill Memory/Disk, durée des tasks, identifier les stragglers

df.rdd.mapPartitionsWithIndex — Inspecter programmatiquement la taille de chaque partition — détecter le skew

Exercices

Ex. 1 — Impact du partitionnement (30 min) `spark.range(500_000)` → afficher le nb de partitions → tester `repartition(2)`, `(16)`, `(200)` → mesurer `groupBy().count()` pour chacun → conclure sur le nb optimal.

Ex. 2 — Détection et correction du skew (35 min) 1M de lignes où 90% valent 'A' dans `categorie`. Observer la distribution post-`groupBy`. Implémenter le salting. Comparer les temps avant/après.

Ex. 3 — Stratégie de cache (25 min) Pipeline 5 étapes (`lecture` → `filtre` → `join` → `agrégation` → `tri`). Mesurer sans cache. Identifier l'étape coûteuse via Spark UI. Cacher et relancer. Comparer `DISK_ONLY` vs `MEMORY_ONLY`.

Ex. 4 — Écriture et lecture optimisées (30 min) Écrire en CSV, Parquet et ORC — comparer les tailles disque. Écrire en Parquet partitionné par `pays` + `annee`. Lire uniquement `France/2024` et vérifier le partition pruning dans `.explain()`.

Pour aller plus loin

Documentation et outils

- Tuning Spark : spark.apache.org/docs/latest/tuning.html
- AQE (Adaptive Query Execution) : spark.apache.org/docs/latest/sql-performance-tuning.html
- Delta Lake (ACID sur Parquet) : delta.io — MERGE, UPDATE, DELETE, time travel

Article fondateur — Parquet / Dremel

Melnik et al. — *Dremel: Interactive Analysis of Web-Scale Datasets*

VLDB 2010 — l'article fondateur du format colonnaire imbriqué qui a inspiré Parquet, le format de référence de l'écosystème Spark/Hadoop.